

has already figured out how, why, and how quickly, an algorithm works; all you have to do to use an algorithm is translate its pseudo-code into something that your system can understand. Because Common Lisp implementations have typically been well implemented and continuously improved upon for decades, they generally employ some of the best and quickest algorithms around for most common tasks.

Good data structures are also necessary for any decent programming language. Data structures are so important that ignoring them will cause any language implementation to slow down to a crawl. Optimising data structures essentially comes down to a concept called locality—which basically says that data that is accessed most frequently should be the fastest to access. Data structures and locality can be observed clearly at almost every level of computing where performance gains have been sought: large sets of CPU registers, memory caches, databases, and caching network proxies, to name a few. Lisp offers a huge set of standard data structures, and they are generally implemented very well.

If Lisp provides such good algorithms and data structures, how is it even possible that Lisp code can be slower than code in other languages? The explanation is based on the most important design decision of Lisp, i.e., general code, a concept otherwise familiar to us as *duality of syntax*. When we write Lisp code, we use as many dualities as possible; the very structure of the language encourages us to do so. Why are Lisp programs usually much shorter than programs in other programming languages? Part of the reason is because any given piece of Lisp code can be used for so much more than a corresponding piece of code in another language, that you re-use it more often. From the perspective of someone more familiar with another programming language, it can feel strange to have to write more to get less, but this is an important Lisp design decision—duality of syntax. The more dualities attached to each expression, the shorter a program seems to be.

Does this mean that to achieve or exceed C's performance, we need to make our Lisp programs as long and dangerous as their corresponding C programs? No, Lisp has macros.

So, enough of selling the concept; let's get down to it, shall we? We'll start with some basics.

Lambda—anonymous function

At times, you'll need a function in only one place in your program. You could create a function with *defun* and call it just once. Sometimes, this is the best thing to do because you can give the function a descriptive name that will help you read the program at some later date. But sometimes the function you need is so trivial or so obvious that you

don't want to have to invent a name, or worry about whether the name might be in use somewhere else. For situations like this, Lisp lets you create an unnamed, or anonymous, function, using the *lambda* form. A *lambda* looks like a *defun* form without the name:

```
(lambda (a b c n)
  (+ (* a (* n n)) (* b n)))
```

You can't evaluate a *lambda* form; it must appear only where Lisp expects to find a function—normally, as the first element of a form:

```
> (lambda (a b c n)
  (+ (* a (* n n)) (* b n)))
#<anonymous interpreted function 21AE8A6A>

> ((lambda (a b c n)
  (+ (* a (* n n)) (* b n)))
  1 3 5 7)
70
```

Macro

The term 'macro' does not have the same meaning in Lisp as it has in other languages. A Lisp macro can be anything from an abbreviation to a compiler for a new language. Macros (in the Lisp sense) are still unique to Lisp. This is partly because in order to have macros, you probably have to make your language look as strange as Lisp. It may also be because if you do add that final increment of power, you can no longer claim to have invented a new language, but only a new dialect of Lisp. If you define a language that has *car*, *cdr*, *cons*, *quote*, *cond*, *atom*, *eq*, and a notation for functions expressed as lists, then you can build all the rest of Lisp out of it. This is, in fact, the defining quality of Lisp: it was in order to make this possible that McCarthy gave Lisp the shape it has.

Because macros can be used to transform Lisp into other programming languages and back, you'd soon discover that all other languages are just skins on top of Lisp. Programming with Lisp is programming at a higher level. Where most languages invent and enforce syntactic and semantic rules, Lisp is general and malleable. With Lisp, you make the rules.

A *defmacro* form looks like a *defun* form. It has a name, a list of argument names, and a body. The macro body returns a form to be evaluated. In other words, you need to write the body of the macro such that it returns a form, not a value. When Lisp evaluates a call to your macro, it first evaluates the body of your macro definition, and then evaluates the result of the first evaluation. By way of comparison, a function's body is evaluated to return a value.